PROYECTO BASE DE DATOS

Tienda informática

Alumno	Joselu
Fecha	Abril 2026
Asignaturas	Bases de Datos · Entornos de Desarrollo
Sistema	tienda_informatica · MySQL 8 · JavaFX

SECCIÓN 1	BASES DE DATOS	ER · Relacional · SQL · Datos iniciales
SECCIÓN 2	ENTORNOS DE DESARROLLO	Clases · Casos de Uso · Secuencia · Colaboración · Actividades

SECCIÓN 1

BASES DE DATOS

Diagrama ER · Diagrama Relacional · SQL · Datos iniciales

1.1 Diagrama Entidad-Relación — Proceso de creación

El modelo conceptual se desarrolló en tres iteraciones. Se partió de la identificación de entidades del negocio, se añadió la especialización disjunta de clientes y finalmente se incorporó el atributo derivado *subtotal* y se ajustaron las cardinalidades.

v1 · Entidades básicas identificadas

Primera versión con las cinco entidades principales y sus relaciones directas. Todavía sin jerarquía de clientes ni atributos derivados.

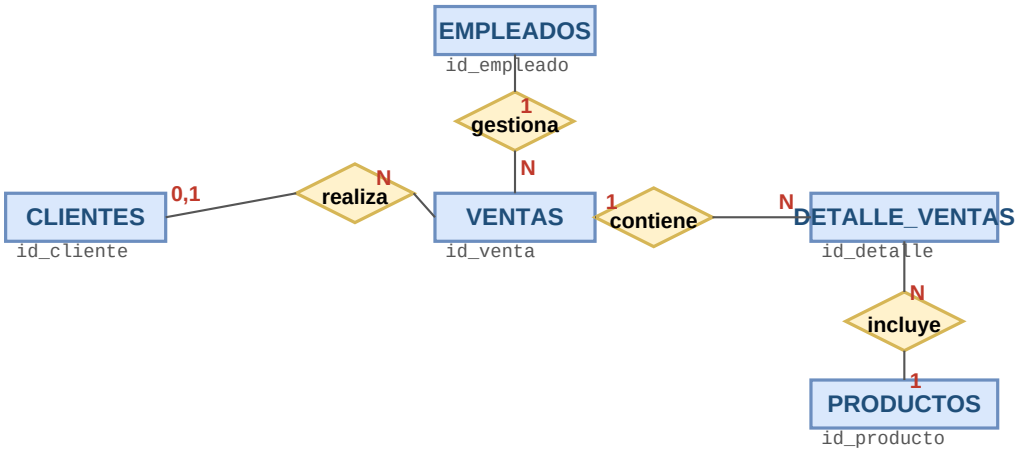

Figura 1a · ER versión 1 — entidades y relaciones básicas

v2 · Incorporación de la herencia de Clientes

Se detectó que los datos de un cliente particular (nombre, apellidos, DNI) difieren completamente de los de una empresa (razón social, CIF, contacto). Se modeló con especialización disjunta (d): CLIENTES como superentidad y CLIENTES_PARTICULAR / CLIENTES_EMPRESA como subtipos.

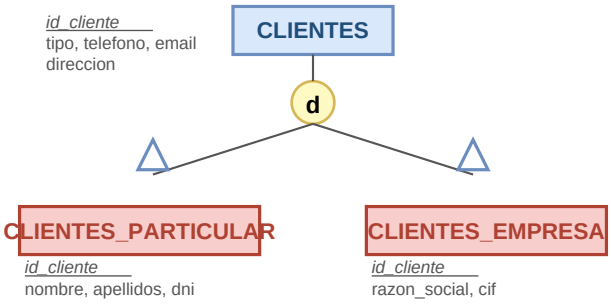

Figura 1b · ER versión 2 — especialización disjunta de Clientes

v3 / Final · Atributo derivado y cardinalidades definitivas

Se añadió *subtotal* como atributo derivado de DETALLE_VENTAS (elipse punteada), se marcó la opcionalidad de la relación cliente-venta (0,1) y se verificaron todas las multiplicidades. Este es el diagrama final entregado.

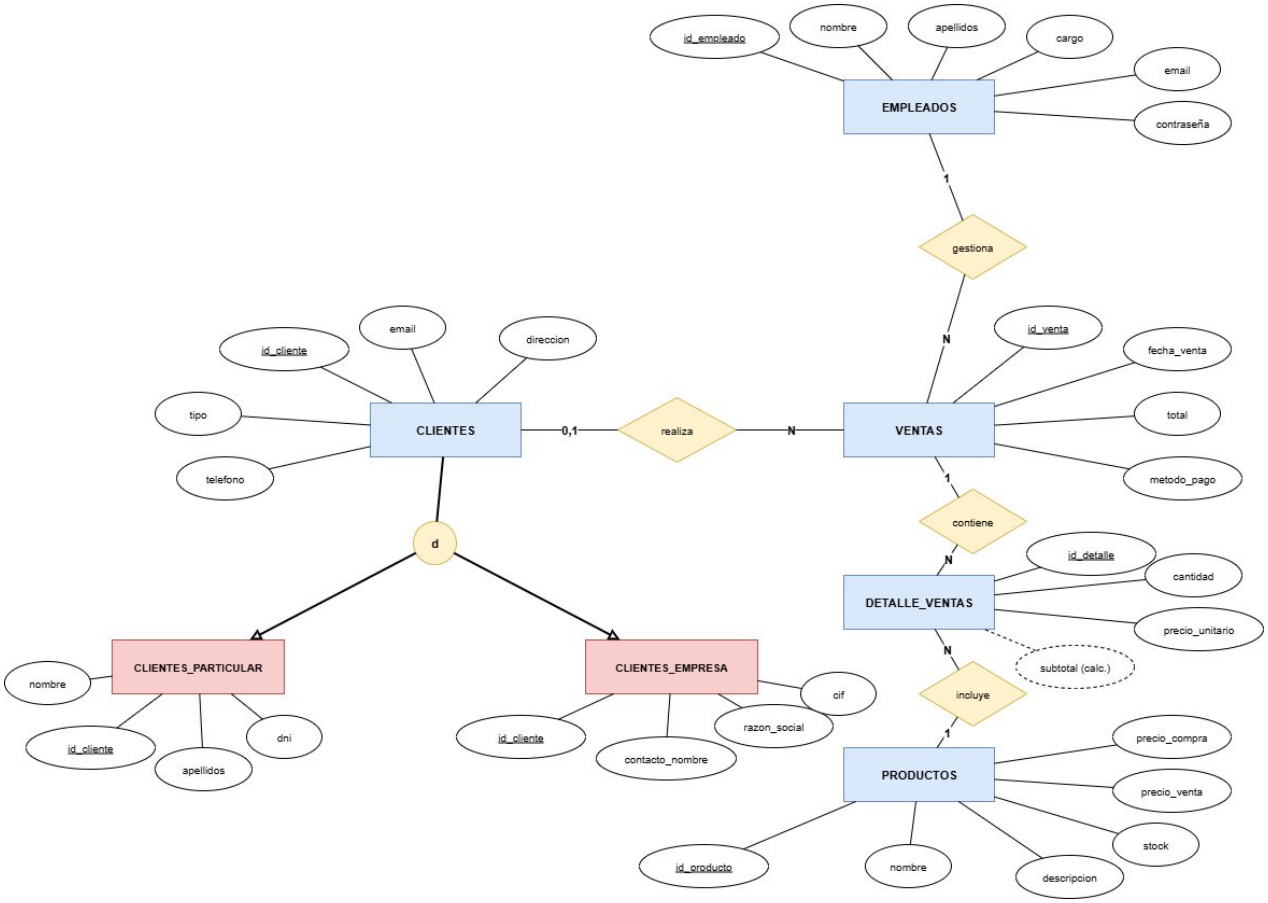

Figura 1c · Diagrama ER final (draw.io) — versión entregable

1.2 Diagrama Relacional

El modelo relacional es la traducción directa del ER a tablas. Se aplicaron las reglas estándar de transformación: entidades fuertes → tablas, herencia disjunta → patrón tabla-por-subtipo, atributo derivado → columna GENERATED ALWAYS.

Tabla	Clave primaria	Claves foráneas	Notas
empleados	id_empleado (AI)	—	ENUM cargo; email UNIQUE
clientes	id_cliente (AI)	—	Discriminador tipo ENUM
clientes_particular	id_cliente	→ clientes (CASCADE)	dni UNIQUE
clientes_empresa	id_cliente	→ clientes (CASCADE)	cif UNIQUE
productos	id_producto (AI)	—	stock INT NOT NULL
ventas	id_venta (AI)	→ clientes (SET NULL), → empleados (CASCADE)	total DECIMAL
detalle_ventas	id_detalle (AI)	→ ventas (CASCADE), → productos	subtotal GENERATED

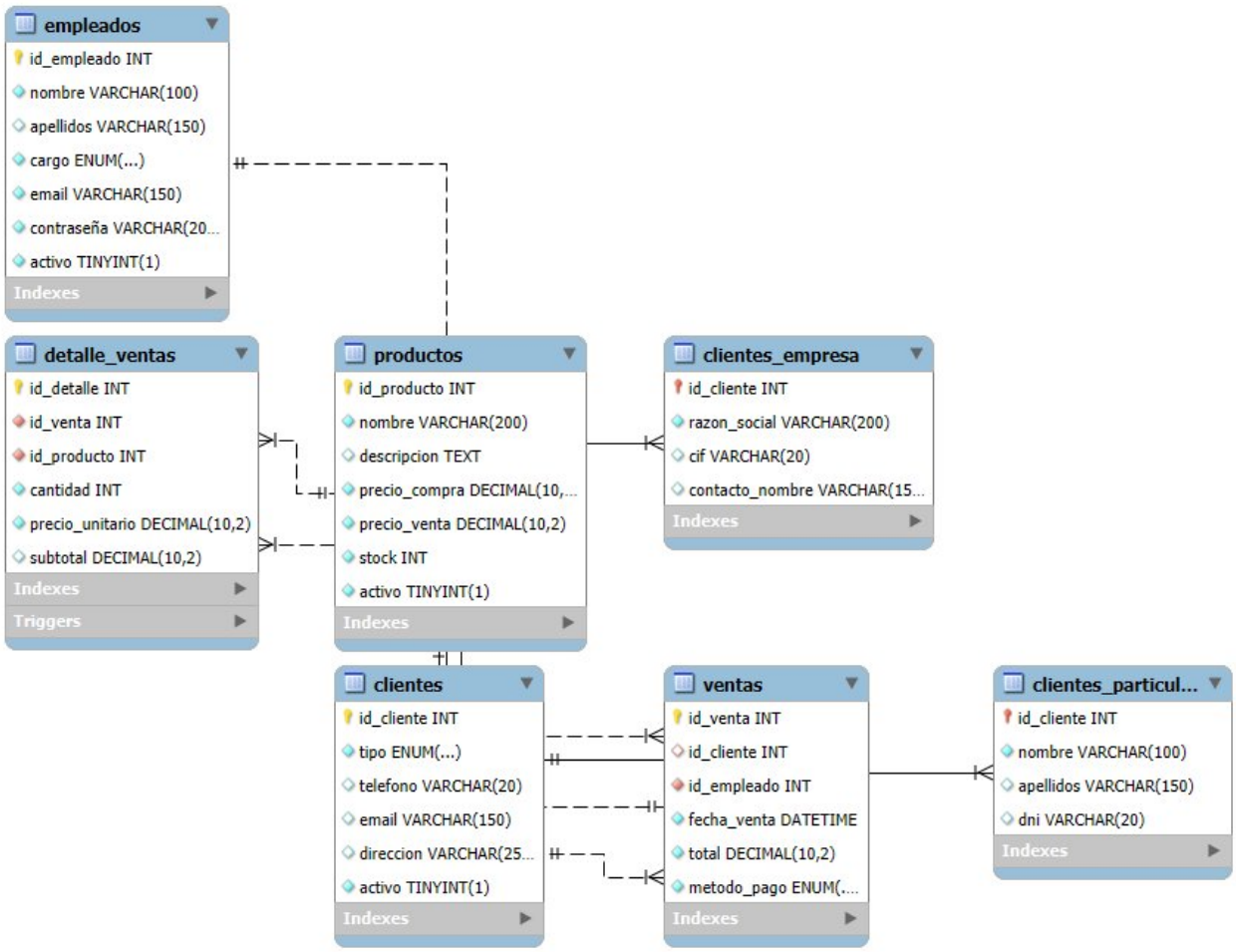

Figura 2 · Diagrama Relacional (MySQL Workbench) — notación crow's foot

1.3 Código SQL — Creación de la base de datos

El DDL implementa la herencia mediante el patrón **tabla-por-subtipo**: la tabla padre *clientes* contiene los atributos comunes; cada subtipo tiene su propia tabla cuya PK es a la vez FK hacia el padre (ON DELETE CASCADE). Se incluyen 4 triggers para integridad de stock y totales, y 2 usuarios MySQL con principio de mínimo privilegio.

Base de datos y tabla productos

```
CREATE DATABASE IF NOT EXISTS tienda_informatica
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE tienda_informatica;

CREATE TABLE IF NOT EXISTS productos (
    id_producto   INT           NOT NULL AUTO_INCREMENT,
    nombre        VARCHAR(200)  NOT NULL,
    descripcion   TEXT,
    precio_compra DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    precio_venta  DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    stock         INT           NOT NULL DEFAULT 0,
    PRIMARY KEY (id_producto)
);
```

Empleados y herencia de clientes (tabla-por-subtipo)

```
CREATE TABLE IF NOT EXISTS empleados (
    id_empleado INT NOT NULL AUTO_INCREMENT,
    nombre      VARCHAR(100) NOT NULL,
    apellidos   VARCHAR(150),
    cargo       ENUM('vendedor','tecnico','admin','gerente') NOT NULL DEFAULT 'vendedor',
    email       VARCHAR(150) NOT NULL UNIQUE,
    contrasena  VARCHAR(200) NOT NULL,
    PRIMARY KEY (id_empleado)
);

-- SUPERCLASE
CREATE TABLE IF NOT EXISTS clientes (
    id_cliente INT NOT NULL AUTO_INCREMENT,
    tipo       ENUM('particular','empresa') NOT NULL,
    telefono   VARCHAR(20),
    email      VARCHAR(150) UNIQUE,
    direccion  VARCHAR(255),
    PRIMARY KEY (id_cliente)
);
-- SUBCLASE 1
CREATE TABLE IF NOT EXISTS clientes_particular (
    id_cliente INT NOT NULL,
    nombre     VARCHAR(100) NOT NULL,
    apellidos  VARCHAR(150),
    dni        VARCHAR(20) UNIQUE,
    PRIMARY KEY (id_cliente),
    CONSTRAINT fk_particular_cliente FOREIGN KEY (id_cliente)
        REFERENCES clientes(id_cliente) ON DELETE CASCADE ON UPDATE CASCADE
);
-- SUBCLASE 2
CREATE TABLE IF NOT EXISTS clientes_empresa (
    id_cliente      INT NOT NULL,
    razon_social    VARCHAR(200) NOT NULL,
    cif             VARCHAR(20) UNIQUE,
    contacto_nombre VARCHAR(150),
    PRIMARY KEY (id_cliente),
    CONSTRAINT fk_empresa_cliente FOREIGN KEY (id_cliente)
        REFERENCES clientes(id_cliente) ON DELETE CASCADE ON UPDATE CASCADE
);
```

Ventas, detalle (atributo GENERATED) y triggers

```
CREATE TABLE IF NOT EXISTS ventas (
    id_venta    INT NOT NULL AUTO_INCREMENT,
    id_cliente  INT,
    id_empleado INT NOT NULL,
    fecha_venta DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    total       DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    metodo_pago ENUM('efectivo','tarjeta','transferencia') NOT NULL DEFAULT 'efectivo',
    PRIMARY KEY (id_venta),
    CONSTRAINT fk_venta_cliente  FOREIGN KEY (id_cliente)
        REFERENCES clientes(id_cliente) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_venta_empleado FOREIGN KEY (id_empleado)
        REFERENCES empleados(id_empleado) ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS detalle_ventas (
    id_detalle      INT NOT NULL AUTO_INCREMENT,
    id_venta        INT NOT NULL,
    id_producto     INT NOT NULL,
    cantidad        INT NOT NULL DEFAULT 1,
    precio_unitario DECIMAL(10,2) NOT NULL,
    subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario) STORED,
    PRIMARY KEY (id_detalle),
    CONSTRAINT fk_detalle_venta    FOREIGN KEY (id_venta)
        REFERENCES ventas(id_venta) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_detalle_producto FOREIGN KEY (id_producto)
        REFERENCES productos(id_producto) ON UPDATE CASCADE
);

DELIMITER $$
-- Trigger: descontar stock y verificar disponibilidad
CREATE TRIGGER trg_stock_al_vender
BEFORE INSERT ON detalle_ventas FOR EACH ROW
BEGIN
    DECLARE v_stock INT;
    SELECT stock INTO v_stock FROM productos WHERE id_producto = NEW.id_producto;
    IF v_stock < NEW.cantidad THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Stock insuficiente';
    END IF;
    UPDATE productos SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto;
END$$
-- Trigger: actualizar total de venta al insertar linea
CREATE TRIGGER trg_actualizar_total_insert
AFTER INSERT ON detalle_ventas FOR EACH ROW
BEGIN
    UPDATE ventas SET total =
        (SELECT SUM(subtotal) FROM detalle_ventas WHERE id_venta = NEW.id_venta)
    WHERE id_venta = NEW.id_venta;
END$$
DELIMITER ;
```

Usuarios MySQL — principio de mínimo privilegio

```
CREATE USER IF NOT EXISTS 'tienda_admin'@'localhost' IDENTIFIED BY 'admin_pass';
GRANT ALL PRIVILEGES ON tienda_informatica.* TO 'tienda_admin'@'localhost';

CREATE USER IF NOT EXISTS 'tienda_app'@'localhost' IDENTIFIED BY 'app_pass';
GRANT SELECT, INSERT, UPDATE ON tienda_informatica.* TO 'tienda_app'@'localhost';

FLUSH PRIVILEGES;
```

1.4 Inserción de datos iniciales para pruebas

Se insertan datos de prueba que cubren todos los escenarios: empleados de distintos roles, cliente particular, cliente empresa, productos variados con precios y stock, y una venta completa con varias líneas de detalle para verificar los triggers de stock y total.

```
-- ■■ PRODUCTOS ■■■
```

```
SELECT v.id_venta, e.nombre AS vendedor,
       COALESCE(cp.nombre, ce.razon_social, 'Anonimo') AS cliente,
       v.fecha_venta, v.total, v.metodo_pago
FROM ventas v
JOIN empleados e ON v.id_empleado = e.id_empleado
LEFT JOIN clientes c ON v.id_cliente = c.id_cliente
LEFT JOIN clientes_particular cp ON c.id_cliente = cp.id_cliente
LEFT JOIN clientes_empresa    ce ON c.id_cliente = ce.id_cliente;
```

SECCIÓN 2

ENTORNOS DE DESARROLLO

Clases UML · Casos de Uso · Secuencia · Colaboración · Actividades

2.1 Diagrama de Clases UML — Proceso de creación

El diagrama de clases modela la capa Java/JavaFX de la aplicación. Se desarrolló en paralelo con el modelo relacional, con tres iteraciones: primero las clases entidad básicas, luego la jerarquía de clientes con clase abstracta, y finalmente los métodos de negocio.

v1 · Clases entidad directas desde las tablas

Primera versión: una clase por tabla, sin herencia, solo atributos y getters/setters básicos.

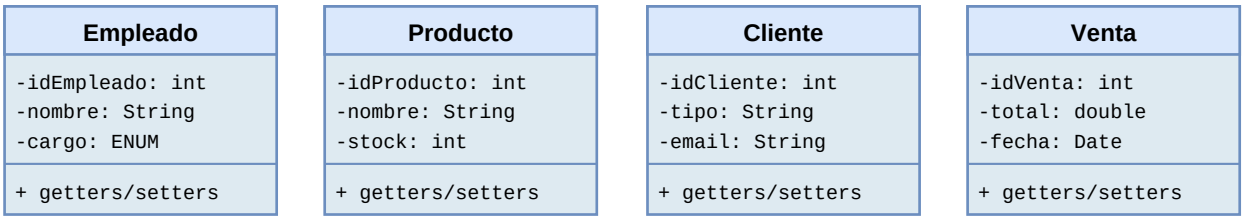

Figura 3a · Clases v1 — clases entidad básicas sin herencia

v2 · Jerarquía Cliente con clase abstracta

Se detectó que Cliente tiene dos especializaciones con atributos distintos. Se convirtió en clase abstracta (Abs. Cliente) con atributos protected, y se crearon ClienteParticular y ClienteEmpresa como subclases concretas.

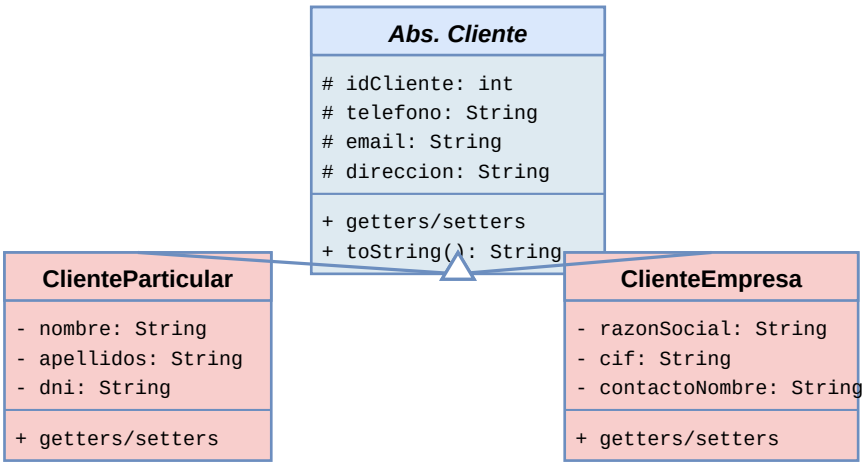

Figura 3b · Clases v2 — jerarquía con clase abstracta Abs. Cliente

Final · Diagrama de Clases completo con relaciones y métodos de negocio

Versión final con todas las clases, sus relaciones UML (asociación, composición, herencia), multiplicidades y métodos de negocio como *calcularTotal()* en Venta y *getSubtotal()* en DetalleVenta.

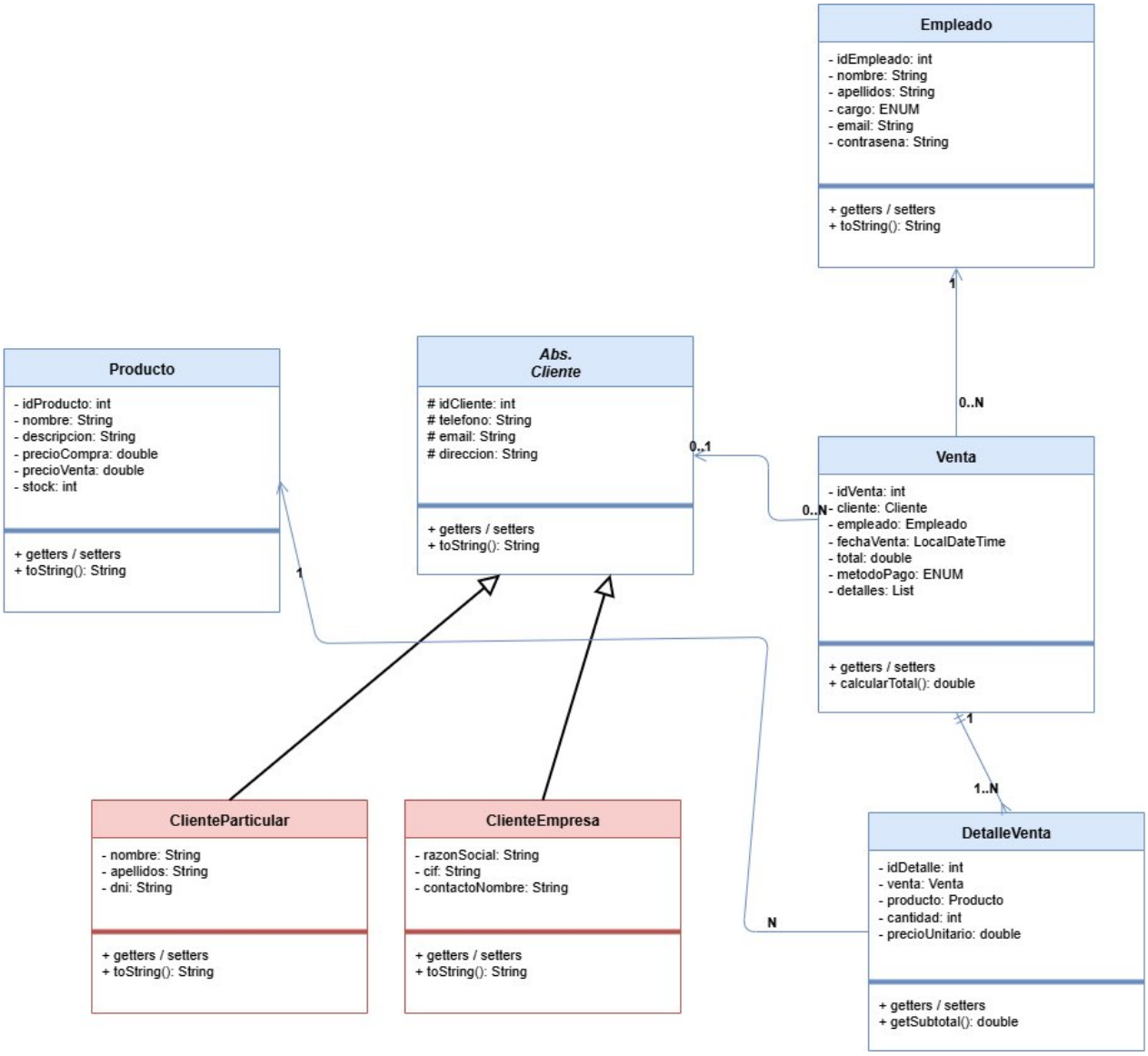

Figura 3c · Diagrama de Clases final (draw.io) — versión entregable

2.2 Diagramas de Casos de Uso — Proceso de creación

Los casos de uso describen las interacciones entre los actores (Vendedor, Técnico, Admin, Cliente, Sistema) y el sistema. Se identificaron en dos fases: primero los casos de uso principales de venta, luego los de gestión administrativa.

v1 · Actores y casos de uso de venta

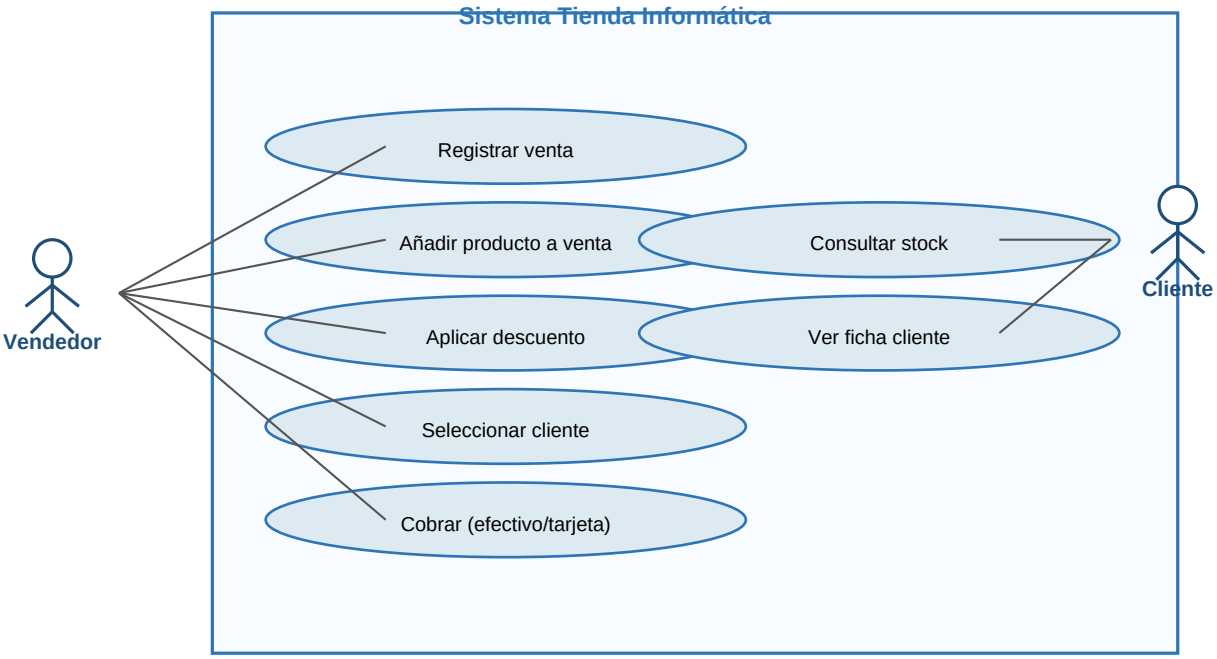

Figura 4a · Casos de Uso v1 — flujo de venta y actores principales

Final · Sistema completo con todos los actores y casos de uso

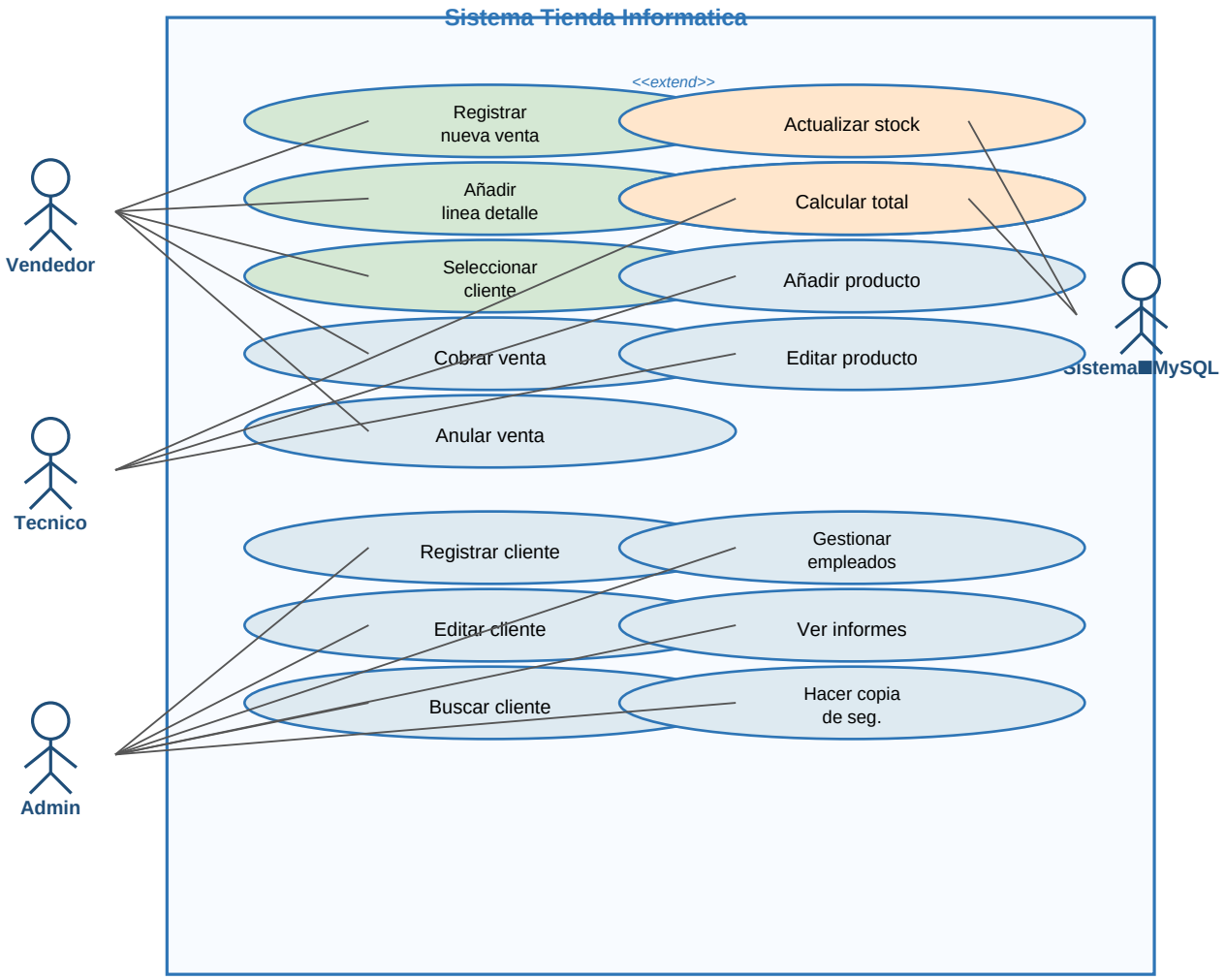

Figura 4b · Diagrama de Casos de Uso final — todos los actores y paquetes

2.3 Diagramas de Secuencia — Casos de uso seleccionados

Los diagramas de secuencia muestran la interacción temporal entre objetos para casos de uso específicos. Se elaboraron para los tres flujos más representativos: registrar venta, registrar cliente y error de stock.

Secuencia 1 · Registrar Venta (flujo normal)

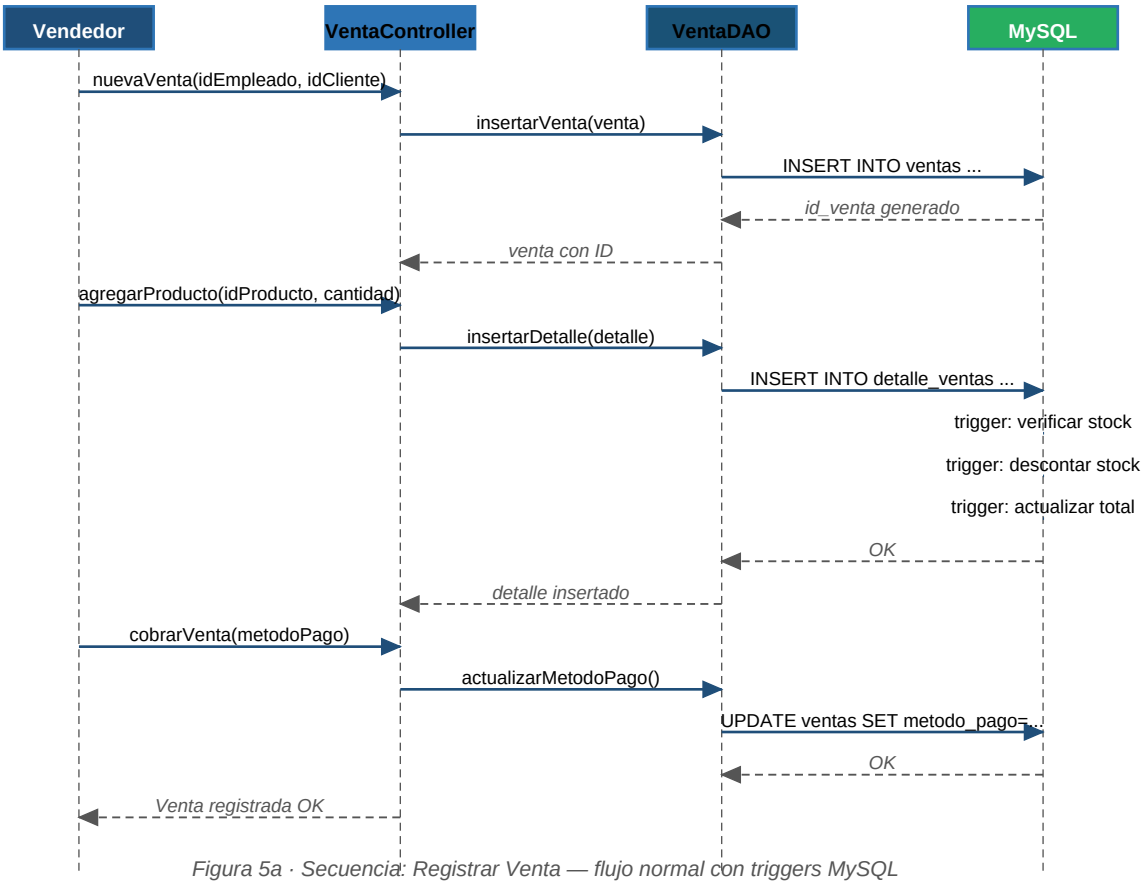

Figura 5a · Secuencia: Registrar Venta — flujo normal con triggers MySQL

Secuencia 2 · Error de Stock insuficiente

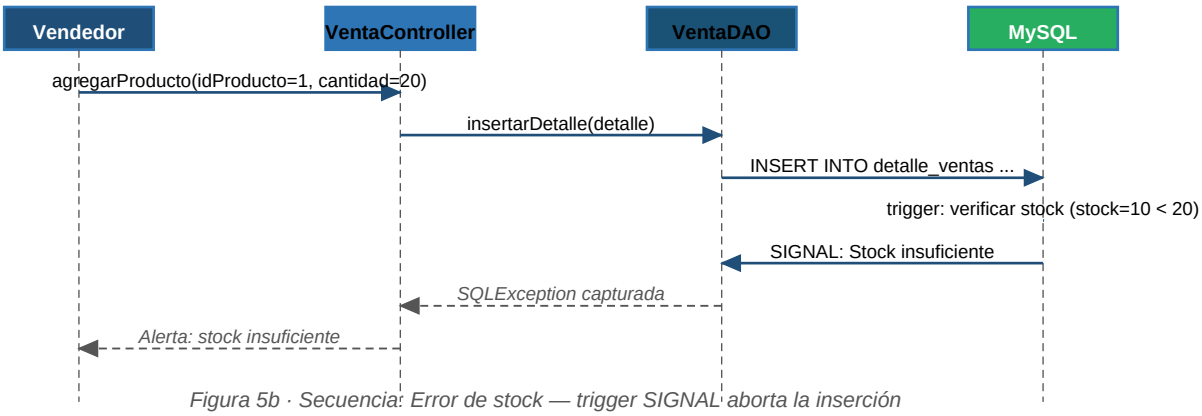

Figura 5b · Secuencia: Error de stock — trigger SIGNAL aborta la inserción

Secuencia 3 · Registrar Cliente Particular

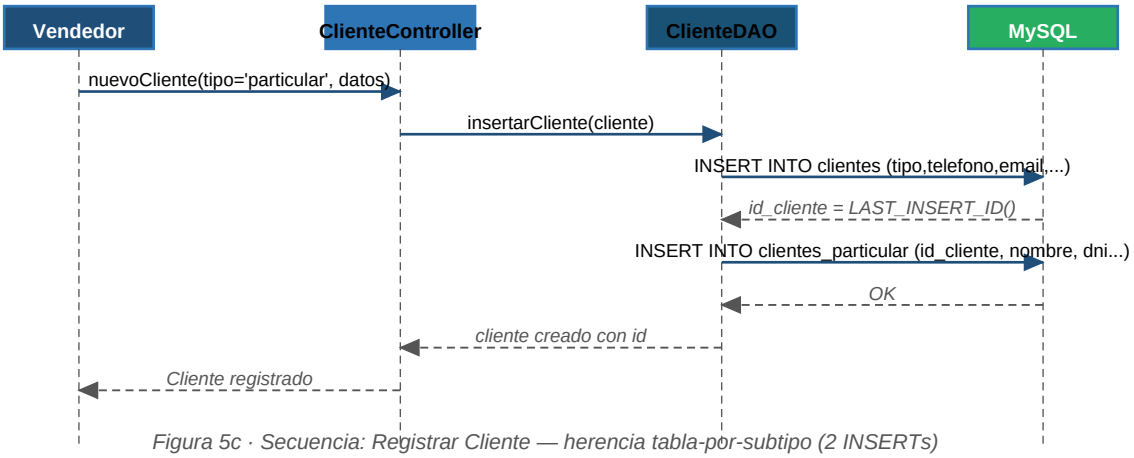

Figura 5c · Secuencia: Registrar Cliente — herencia tabla-por-subtipo (2 INSERTs)

2.4 Diagramas de Colaboración — Relación entre objetos

Los diagramas de colaboración muestran los mismos flujos que los de secuencia pero enfatizando las relaciones estructurales entre objetos y el orden de los mensajes. Se representan para dos casos de uso.

Colaboración 1 · Registrar Venta

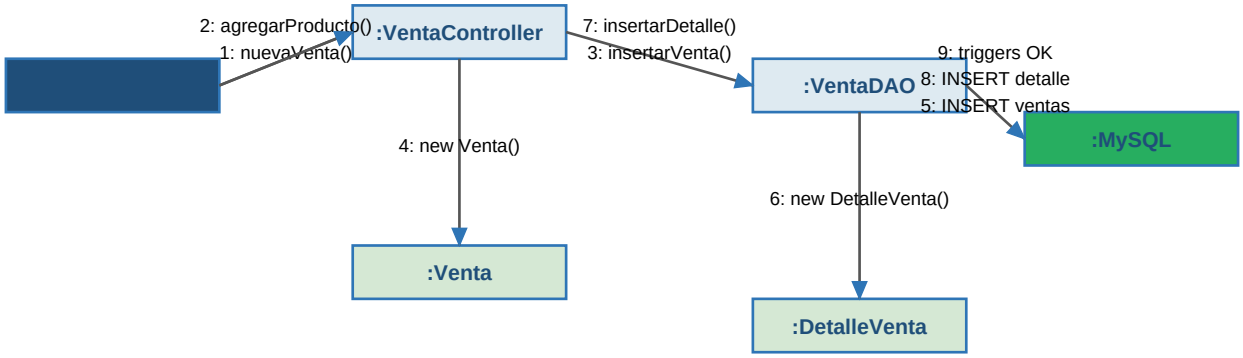

Figura 6a · Colaboración: Registrar Venta — mensajes numerados entre objetos

Colaboración 2 · Registrar Cliente (herencia)

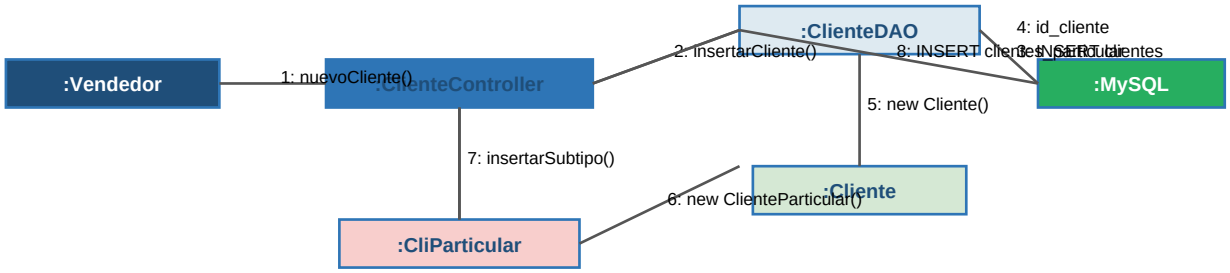

Figura 6b · Colaboración: Registrar Cliente — doble INSERT por herencia tabla-subtipo

2.5 Diagramas de Actividades — Flujo de trabajo del sistema

Los diagramas de actividades modelan el flujo completo de las operaciones más importantes, incluyendo decisiones, condiciones de error y swimlanes (carriles) por actor.

Actividad 1 · Proceso completo de Venta

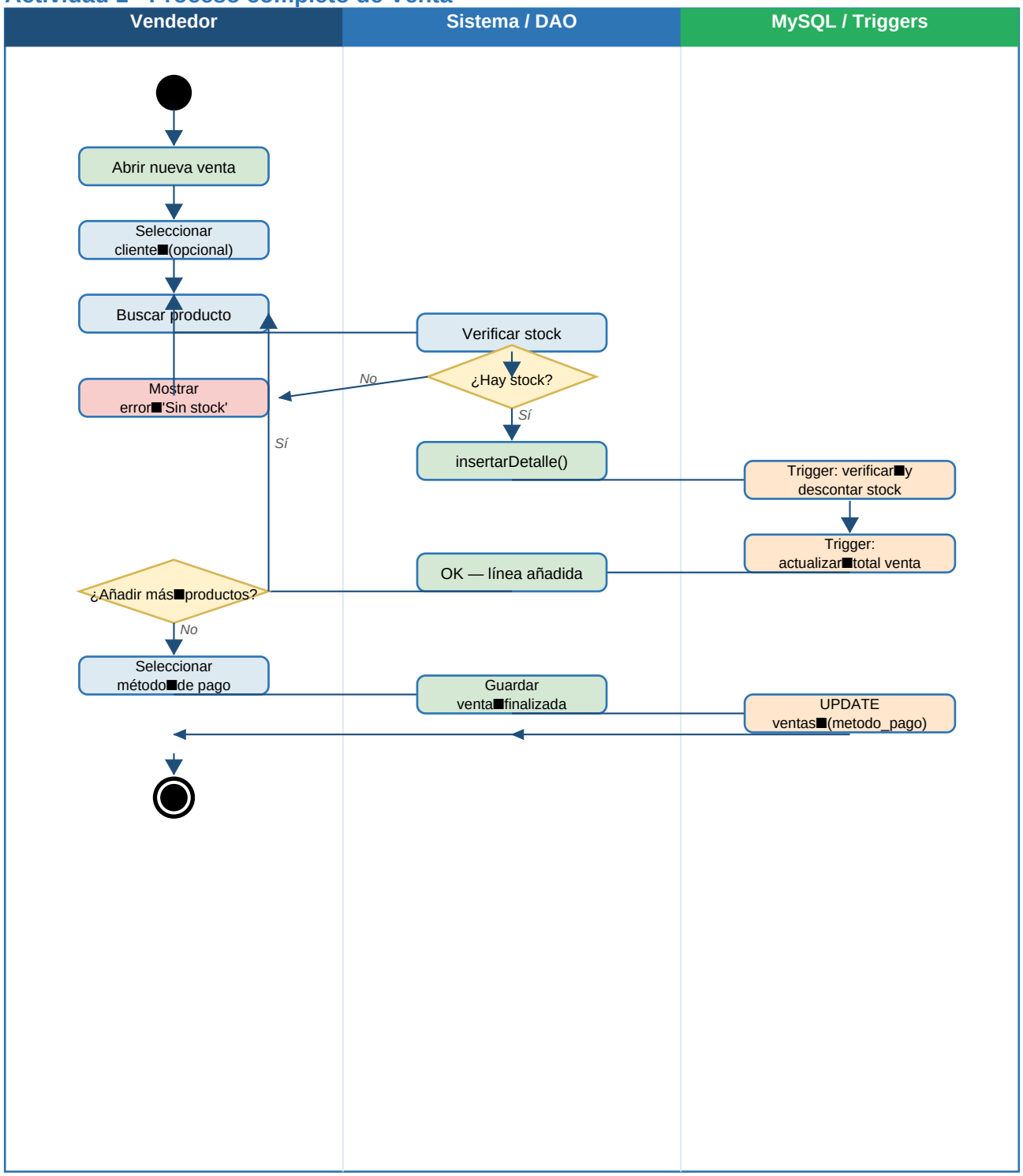

Figura 7a · Actividad: Proceso completo de Venta (swimlanes: Vendedor / Sistema / MySQL)

Actividad 2 · Registrar nuevo Cliente

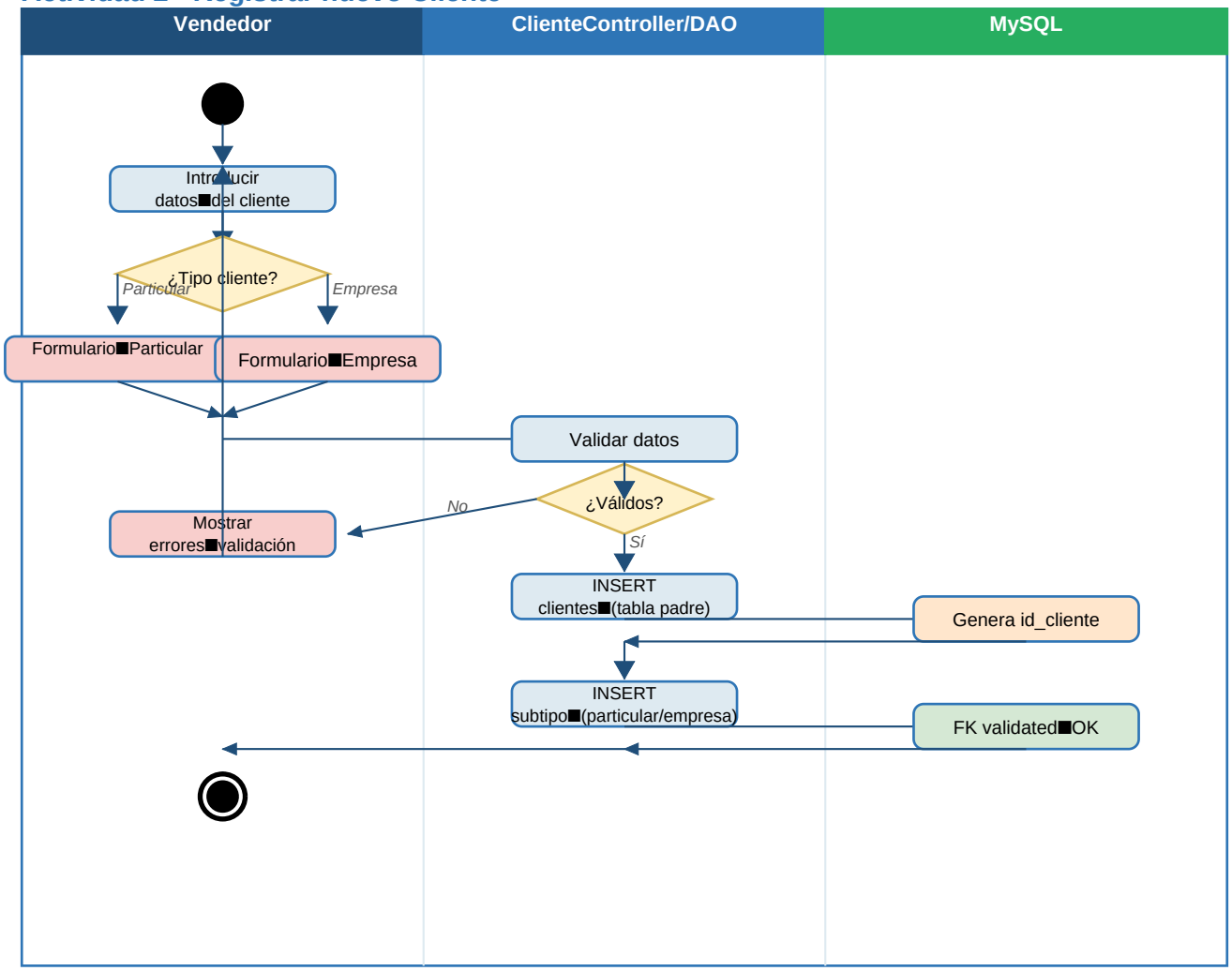

Figura 7b · Actividad: Registrar Cliente — decisión de tipo y doble INSERT por herencia

Actividad 3 · Login y control de acceso por rol

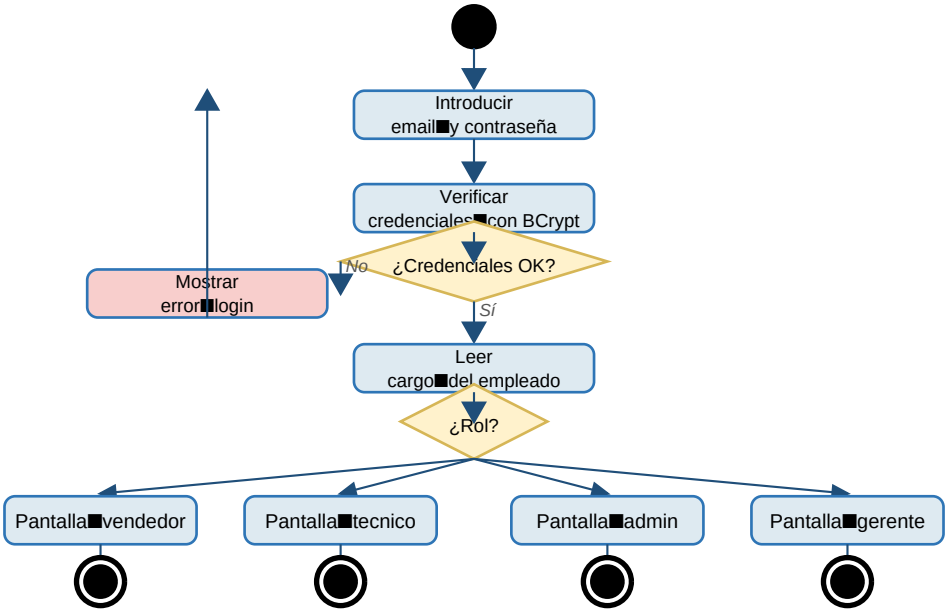

Figura 7c · Actividad: Login y selección de pantalla según rol del empleado

Resumen del proyecto

Artefacto	Estado	Herramienta
Diagrama ER	✓ **Entregado**	draw.io
Diagrama Relacional	✓ **Entregado**	MySQL Workbench
SQL creación BD (herencia)	✓ **Entregado**	MySQL 8
Datos iniciales SQL	✓ **Entregado**	MySQL 8
Diagrama de Clases UML	✓ **Entregado**	draw.io
Casos de Uso	✓ **Generado**	draw.io / ReportLab
Diagrama de Secuencia (3)	✓ **Generado**	ReportLab
Diagrama de Colaboración (2)	✓ **Generado**	ReportLab
Diagrama de Actividades (3)	✓ **Generado**	ReportLab